

Linked list

Linked List

- A linked list is a data structure used to store a collection of elements.
- Each element, called a node, contains the data and a reference (or pointer) to the next node in the sequence.
- This structure allows for efficient insertions and deletions since you only need to update the pointers.

Reference: [Dev Frank](https://medium.com/@Dev_Frank/c-linked-list-singly-linked-list-12aecb168834), **C-LINKED LIST (Singly Linked List)** Medium

https://medium.com/@Dev_Frank/c-linked-list-singly-linked-list-12aecb168834

Types of Linked Lists

- **Singly Linked List:** Each node points to the next node. The last node points to **NULL**.
- **Doubly Linked List:** Each node points to both the next and the previous node.
- **Circular Linked List:** The last node points back to the first node, forming a circle.

Node Definition

1. **Node:** A node is the basic building block of a linked list. Each node contains data and a pointer to the next node.
 - **Data:** This is the value or information the node holds.
 - **Pointer:** This is like an address that tells you where the next node in the list is.
2. **Head:** This a pointer or reference to the first node in the list.
 - It serves as the starting point for any operations that involve traversing or manipulating the list.
 - Without the head, it would be impossible to access the rest of the nodes in the list.

```
// Define the node structure
struct Node {
    int data;           // Data part
    struct Node* next; // Pointer to the next node
};
```

1. Creation of a Node:

- You need a function to create and initialize a new node.
- Allocating memory for a new node.
- Setting the node's data and initializing the **next** pointer to **NULL**.

```
#include <stdio.h>
#include <stdlib.h>

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (!newNode) {
        printf("Memory allocation error\n");
        return NULL;
    }
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

2. Inserting Nodes

Nodes can be inserted at the beginning, at the end, or at a specific position.

— **insertAtBeginning** function creates a new node, sets its **next** pointer to the current head, and updates the head to the new node.

```
void insertAtBeginning(struct Node** headRef, int data) {  
    struct Node* newNode = createNode(data);  
    newNode->next = *headRef;  
    *headRef = newNode;  
}
```

2. Inserting Nodes

— **insertAtEnd** function creates a new node, sets its **next** pointer to the current head, and updates the head to the new node.

```
void insertAtEnd(struct Node** headRef, int data) {  
    struct Node* newNode = createNode(data);  
    if (*headRef == NULL) {  
        *headRef = newNode;  
    } else {  
        struct Node* temp = *headRef;  
        while (temp->next != NULL) {  
            temp = temp->next;  
        }  
        temp->next = newNode;  
    }  
}
```

3. Deletion :

Deletion can occur at various positions in the list: at the beginning, at the end, or at a specific position.

Delete from the Beginning

To delete the first node, you simply move the head pointer to the next node.

```
void deleteFromBeginning(struct Node** headRef) {  
    if (*headRef == NULL) {  
        printf("List is empty\n");  
        return;  
    }  
    struct Node* temp = *headRef;  
    *headRef = (*headRef)->next;  
    free(temp);  
}
```

- Checks if the list is empty. If it is, it prints a message.
- If not, it updates the head to point to the next node and frees the memory of the original first node.

3. Deletion :

Delete from the End : To delete the last node, you need to traverse the list to find the second last node and update its **next** pointer to **NULL**.

```
void deleteFromEnd(struct Node** headRef) {
    if (*headRef == NULL) {
        printf("List is empty\n");
        return;
    }
    if ((*headRef)->next == NULL) {
        free(*headRef);
        *headRef = NULL;
        return;
    }
    struct Node* temp = *headRef;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }
    free(temp->next);
    temp->next = NULL;
}
```

- Checks if the list is empty. If it is, it prints a message.
- If the list has only one node, it frees it and sets the head to **NULL**.
- Otherwise, it traverses the list to find the second last node, updates its **next** pointer to **NULL**, and frees the last node.

3. Deletion :

Delete from a Specific Position

To delete a node at a specific position, you need to find the node just before that position and update its next pointer

- Checks if the list is empty. If it is, it prints a message.
- If the position is 0, it updates the head to point to the second node and frees the first node.
- Otherwise, it traverses the list to find the node just before the position, updates its **next** pointer to skip the node to be deleted, and frees the node.

```
void deleteFromPosition(struct Node** headRef, int position) {
    if (*headRef == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node* temp = *headRef;

    if (position == 0) {
        *headRef = temp->next;
        free(temp);
        return;
    }

    for (int i = 0; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
    }

    if (temp == NULL || temp->next == NULL) {
        printf("Position does not exist\n");
        return;
    }

    struct Node* next = temp->next->next;
    free(temp->next);
    temp->next = next;
}
```

Main function

```
int main() {  
    struct Node* head = NULL;  
  
    // Insert nodes  
    insertAtBeginning(&head, 10);  
    insertAtBeginning(&head, 20);  
    insertAtBeginning(&head, 30);  
  
    // Print list  
    printf("Initial list: ");  
    printList(head);  
  
    // Delete from beginning  
    deleteFromBeginning(&head);  
    printf("After deleting from beginning: ");  
    printList(head);  
  
    // Delete from end  
    deleteFromEnd(&head);  
    printf("After deleting from end: ");  
    printList(head);  
  
    // Delete from a specific position  
    insertAtBeginning(&head, 40);  
    insertAtBeginning(&head, 50);  
    deleteFromPosition(&head, 1);  
    printf("After deleting from position 1: ");  
    printList(head);  
  
    return 0;  
}
```

4. Traversing the List :

Starting from the head, you follow the pointers from one node to the next until you reach the end.

- Starts at the head node.
- Iterates through the list using a temporary pointer (**temp**).
- Prints the data of each node
- Stops when **temp** becomes **NULL**, indicating the end of the list.

```
// Function to traverse the list and print the elements
void traverseList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

Doubly Linked List and operations

- An integer data value.
- A pointer to the next node.
- A pointer to the previous node.

```
typedef struct Node {  
    int data;  
    struct Node* next;  
    struct Node* prev;  
} Node;
```

- Insert at the Beginning
- Insert at the End
- Delete a Node
- Display List

Creation of a Node

```
Node* createNode(int data) {  
    Node* newNode = (Node*)malloc(sizeof(Node));  
    newNode->data = data;  
    newNode->next = NULL;  
    newNode->prev = NULL;  
    return newNode;  
}
```

Insertion :: At Front

```
void insertAtFront(Node** head, int data) {  
    Node* newNode = createNode(data);  
    newNode->next = *head;  
    if (*head != NULL) {  
        (*head)->prev = newNode;  
    }  
    *head = newNode;  
}
```

Insertion :: At End

```
void insertAtEnd(Node** head, int data) {  
    Node* newNode = createNode(data);  
    if (*head == NULL) {  
        *head = newNode;  
        return;  
    }  
    Node* temp = *head;  
    while (temp->next != NULL) {  
        temp = temp->next;  
    }  
    temp->next = newNode;  
    newNode->prev = temp;  
}
```

Inserting a Node After a Given Node:

```
void insertAfter(Node* prevNode, int data) {  
    if (prevNode == NULL) {  
        printf("The given previous node cannot be NULL\n");  
        return;  
    }  
  
    Node* newNode = createNode(data);  
    newNode->next = prevNode->next;  
    newNode->prev = prevNode;  
  
    if (prevNode->next != NULL) {  
        prevNode->next->prev = newNode;  
    }  
  
    prevNode->next = newNode;  
}
```

Inserting a Node Before a Given Node:

```
void insertBefore(Node** head, Node* nextNode, int data) {
    if (nextNode == NULL) {
        printf("The given next node cannot be NULL\n");
        return;
    }

    Node* newNode = createNode(data);
    newNode->prev = nextNode->prev;
    newNode->next = nextNode;

    if (nextNode->prev != NULL) {
        nextNode->prev->next = newNode;
    } else {
        // If the new node is inserted before the head node
        *head = newNode;
    }

    nextNode->prev = newNode;
}
```

Traverse

```
void traverseForward(Node* head) {  
    Node* temp = head;  
    while (temp != NULL) {  
        printf("%d ", temp->data);  
        temp = temp->next;  
    }  
    printf("\n");  
}
```

```
void traverseBackward(Node* head) {  
    if (head == NULL) {  
        return;  
    }  
  
    Node* temp = head;  
    while (temp->next != NULL) {  
        temp = temp->next;  
    }  
  
    while (temp != NULL) {  
        printf("%d ", temp->data);  
        temp = temp->prev;  
    }  
    printf("\n");  
}
```

Deletion:: From the Front: Remove a node from the beginning of the list

```
void deleteFromFront(Node** head) {  
    if (*head == NULL) {  
        printf("The list is already empty\n");  
        return;  
    }  
  
    Node* temp = *head;  
    *head = (*head)->next;  
  
    if (*head != NULL) {  
        (*head)->prev = NULL;  
    }  
  
    free(temp);  
}
```

Deletion::From the End: Remove a node from the end of the list.

```
void deleteFromEnd(Node** head) {
    if (*head == NULL) {
        printf("The list is already empty\n");
        return;
    }

    Node* temp = *head;

    // Traverse to the last node
    while (temp->next != NULL) {
        temp = temp->next;
    }

    // If there's only one node in the list
    if (temp->prev == NULL) {
        *head = NULL;
    } else {
        temp->prev->next = NULL;
    }

    free(temp);
}
```

Deletion::Given Node: Remove a specific node from the list.

```
void deleteGivenNode(Node** head, Node* del) {
    if (*head == NULL || del == NULL) {
        printf("The list or the given node is NULL\n");
        return;
    }

    if (*head == del) {
        *head = del->next;
    }

    if (del->next != NULL) {
        del->next->prev = del->prev;
    }

    if (del->prev != NULL) {
        del->prev->next = del->next;
    }

    free(del);
}
```

Traversal::Forward Traversal: Visit each node starting from the head and moving to the tail.

```
void traverseForward(Node* head) {  
    Node* temp = head;  
    while (temp != NULL) {  
        printf("%d ", temp->data);  
        temp = temp->next;  
    }  
    printf("\n");  
}
```

Traversal::Backward Traversal: Visit each node starting from the tail and moving to the head.

```
void traverseBackward(Node* head) {  
    Node* temp = head;  
    if (temp == NULL) {  
        return;  
    }  
    while (temp->next != NULL) {  
        temp = temp->next;  
    }  
    while (temp != NULL) {  
        printf("%d ", temp->data);  
        temp = temp->prev;  
    }  
    printf("\n");  
}
```

Search

```
Node* searchNode(Node* head, int target) {  
    Node* current = head;  
    while (current != NULL) {  
        if (current->data == target) {  
            return current;  
        }  
        current = current->next;  
    }  
    return NULL; // Node not found  
}
```

Update

```
void updateNode(Node* head, int oldData, int newData) {  
    Node* current = head;  
    while (current != NULL) {  
        if (current->data == oldData) {  
            current->data = newData;  
            return;  
        }  
        current = current->next;  
    }  
    printf("Node with data %d not found.\n", oldData);  
}
```

Reverse

```
void reverse(Node** head) {  
    Node* temp = NULL;  
    Node* current = *head;  
  
    while (current != NULL) {  
        temp = current->prev;  
        current->prev = current->next;  
        current->next = temp;  
        current = current->prev;  
    }  
  
    if (temp != NULL) {  
        *head = temp->prev;  
    }  
}
```

Circular Linked List and operations

Singly Circular Linked List : The last node in the single linked list points back to the **first node**, forming a loop.

Doubly Circular Linked List: Each node has two pointers (next and previous). The last node points forward to the first node **and** the first node points backward to the last node.

Circular Linked List:: Node definition

```
// Define the node structure
struct Node {
    int data;           // Data part
    struct Node* next; // Pointer to the next node
};
```

Circular Linked List:: Node creation

```
#include <stdio.h>
#include <stdlib.h>

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (!newNode) {
        printf("Memory allocation error\n");
        return NULL;
    }
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

Circular Linked List:: insertion at beginning

```
void insertAtBeginning(struct Node** headRef, int data) {  
    struct Node* newNode = createNode(data);  
    newNode->next = *headRef;  
    *headRef = newNode;  
}
```

This is for Single linked list

Can you say where you need to change in the above code to make it circular linked list?

```

struct Node* insertAtBeginning(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;

    // If list is empty
    if (head == NULL) {
        newNode->next = newNode;    // Points to itself
        head = newNode;
    } else {
        struct Node* temp = head;

        // Traverse to the last node
        while (temp->next != head) {
            temp = temp->next;
        }

        newNode->next = head;    // New node points to old head
        temp->next = newNode;    // Last node points to new node
        head = newNode;        // Update head
    }

    return head;
}

```

Circular Linked List:: insertion at beginning

Circular Linked List:: insertion at end

```
void insertAtEnd(struct Node** headRef, int data) {  
    struct Node* newNode = createNode(data);  
    if (*headRef == NULL) {  
        *headRef = newNode;  
    } else {  
        struct Node* temp = *headRef;  
        while (temp->next != NULL) {  
            temp = temp->next;  
        }  
        temp->next = newNode;  
    }  
}
```

This is for singled linked list.

Think how to make it for
circular linked list?

```

struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;

    // If list is empty
    if (head == NULL) {
        newNode->next = newNode; // Point to itself
        return newNode;
    }

    struct Node* temp = head;

    // Traverse to the last node
    while (temp->next != head) {
        temp = temp->next;
    }

    temp->next = newNode; // Last node points to new node
    newNode->next = head; // New node points to head

    return head; // Head remains unchanged
}

```



Circular Linked List:: insertion at end

```
void deleteFromBeginning(struct Node** headRef) {  
    if (*headRef == NULL) {  
        printf("List is empty\n");  
        return;  
    }  
    struct Node* temp = *headRef;  
    *headRef = (*headRef)->next;  
    free(temp);  
}
```

single Linked
List:: Delete
from beginning

```

struct Node* deleteFromBeginning(struct Node* head) {
    // If list is empty
    if (head == NULL) {
        printf("List is empty\n");
        return NULL;
    }

    // If only one node exists
    if (head->next == head) {
        free(head);
        return NULL;
    }

    struct Node* temp = head;
    struct Node* last = head;

    // Find the last node
    while (last->next != head) {
        last = last->next;
    }

    head = head->next;    // Move head to next node
    last->next = head;    // Last node points to new head
    free(temp);          // Free old head

    return head;
}

```

Circular
Linked List::
delete from
beginning

```
void deleteFromEnd(struct Node** headRef) {  
    if (*headRef == NULL) {  
        printf("List is empty\n");  
        return;  
    }  
    if ((*headRef)->next == NULL) {  
        free(*headRef);  
        *headRef = NULL;  
        return;  
    }  
    struct Node* temp = *headRef;  
    while (temp->next->next != NULL) {  
        temp = temp->next;  
    }  
    free(temp->next);  
    temp->next = NULL;  
}
```

Single Linked List:: Delete from end

```

struct Node* deleteFromEnd(struct Node* head) {
    // If list is empty
    if (head == NULL) {
        printf("List is empty\n");
        return NULL;
    }

    // If only one node
    if (head->next == head) {
        free(head);
        return NULL;
    }

    struct Node* temp = head;
    struct Node* prev = NULL;

    // Traverse to last node
    while (temp->next != head) {
        prev = temp;
        temp = temp->next;
    }

    // temp = last node
    // prev = second last node
    prev->next = head; // Make second last point to head
    free(temp);        // Delete last node

    return head;        // Head remains unchanged
}

```

Circular Linked List:: delete from end

```

void deleteFromPosition(struct Node** headRef, int position) {
    if (*headRef == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node* temp = *headRef;

    if (position == 0) {
        *headRef = temp->next;
        free(temp);
        return;
    }

    for (int i = 0; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
    }

    if (temp == NULL || temp->next == NULL) {
        printf("Position does not exist\n");
        return;
    }

    struct Node* next = temp->next->next;
    free(temp->next);
    temp->next = next;
}

```

Single Linked List:: Delete from a specific position

Circular Linked List:: delete from a specific position

```
struct Node* deleteFromPosition(struct Node* head, int position) {
    if (head == NULL) {
        printf("List is empty\n");
        return NULL;
    }

    // If position is 1 → delete from beginning
    if (position == 1) {
        // Only one node
        if (head->next == head) {
            free(head);
            return NULL;
        }

        struct Node* last = head;
        while (last->next != head) {
            last = last->next;
        }

        struct Node* temp = head;
        head = head->next;
        last->next = head;
        free(temp);

        return head;
    }
}
```

```
// Delete from position other than 1
struct Node* temp = head;
struct Node* prev = NULL;
int count = 1;

while (count < position && temp->next != head) {
    prev = temp;
    temp = temp->next;
    count++;
}

// If position is invalid
if (count != position) {
    printf("Invalid position\n");
    return head;
}

prev->next = temp->next;
free(temp);

return head;
}
```

Problem discussion on Linked Lists

```
int getMiddle(struct Node* head) {  
  
    struct Node* slowptr = head;  
    struct Node* fastptr = head;  
  
    // traverse the linked list  
    while (fastptr != NULL && fastptr->next != NULL) {  
  
        // move the fast pointer by two nodes  
        fastptr = fastptr->next->next;  
  
        // move the slow pointer by one node  
        slowptr = slowptr->next;  
    }  
  
    return slowptr->data;  
}
```

Middle Node in a Linked List:

Problem discussion on Linked Lists

```
int detectLoop(struct Node* head) {  
  
    // Fast and slow pointers initially points to the head  
    struct Node *slow = head, *fast = head;  
  
    // Loop that runs while fast and slow pointer are not  
    // nullptr and not equal  
    while (slow && fast && fast->next) {  
        slow = slow->next;  
        fast = fast->next->next;  
  
        // If fast and slow pointer points to the same node,  
        // then the cycle is detected  
        if (slow == fast) {  
            return true;  
        }  
    }  
    return false;  
}
```

Detect Cycle in Linked List

```

// Function to merge two linked lists
struct Node *merge(struct Node *head1, struct Node *head2) {
    struct Node *curr1 = head1;
    struct Node *curr2 = head2;
    struct Node *next1, *next2;

    // Traverse both lists and merge them
    while (curr1 != NULL && curr2 != NULL) {

        // Save the next nodes of the current
        // nodes in both lists
        next1 = curr1->next;
        next2 = curr2->next;

        // Insert the current node from the second list
        // after the current node from the first list
        curr2->next = curr1->next;
        curr1->next = curr2;

        // Update the pointers for the next iteration
        curr1 = next1;
        curr2 = next2;
    }

    // Return the remaining part of the second list
    return curr2;
}

```

Problem discussion on Linked Lists

**Merge a linked list into
another linked list at
alternate positions**

Polynomial manipulation using Linked List

Polynomial manipulation using Linked List

```
struct Node {  
    int coeff;  
    int power;  
    struct Node* next;  
};
```

```
/* Create new node */  
Node* createNode(int coeff, int power) {  
    Node* newNode = (Node*)malloc(sizeof(Node));  
    newNode->coeff = coeff;  
    newNode->power = power;  
    newNode->next = NULL;  
    return newNode;  
}
```

```

Node* addPolynomials(Node* poly1, Node* poly2) {
    Node* result = NULL;

    while (poly1 != NULL && poly2 != NULL) {
        if (poly1->power > poly2->power) {
            result = insertTerm(result, poly1->coeff, poly1->power);
            poly1 = poly1->next;
        } else if (poly1->power < poly2->power) {
            result = insertTerm(result, poly2->coeff, poly2->power);
            poly2 = poly2->next;
        } else {
            result = insertTerm(result,
                                poly1->coeff + poly2->coeff,
                                poly1->power);
            poly1 = poly1->next;
            poly2 = poly2->next;
        }
    }

    while (poly1 != NULL) {
        result = insertTerm(result, poly1->coeff, poly1->power);
        poly1 = poly1->next;
    }

    while (poly2 != NULL) {
        result = insertTerm(result, poly2->coeff, poly2->power);
        poly2 = poly2->next;
    }

    return result;
}

```

Polynomial
manipulation using
Linked List

```

/* Insert term in descending order of power */
Node* insertTerm(Node* head, int coeff, int power) {
    Node* newNode = createNode(coeff, power);

    if (head == NULL || power > head->power) {
        newNode->next = head;
        return newNode;
    }

    Node* temp = head;

    while (temp->next != NULL && temp->next->power > power) {
        temp = temp->next;
    }

    if (temp->next != NULL && temp->next->power == power) {
        temp->next->coeff += coeff;
        free(newNode);
    } else {
        newNode->next = temp->next;
        temp->next = newNode;
    }

    return head;
}

```

Polynomial manipulation using Linked List



Rotate a linked list k positions left/right

```
function rotate(head, k) {  
    if (k === 0 || head === null) {  
        return head;  
    }  
  
    // rotate the list by k nodes  
    for (let i = 0; i < k; ++i) {  
        let curr = head;  
        while (curr.next !== null) {  
            curr = curr.next;  
        }  
  
        // move the first node to the last  
        curr.next = head;  
        curr = curr.next;  
        head = head.next;  
        curr.next = null;  
    }  
    return head;  
}
```

```

struct Pair splitList(struct Node* head) {
    struct Node* slow = head;
    struct Node* fast = head;
    struct Pair result;

    if (head == NULL) {
        result.first = NULL;
        result.second = NULL;
        return result;
    }

    // For odd nodes, fast->next is head and
    // for even nodes, fast->next->next is head
    while (fast->next != head
    && fast->next->next != head) {
        fast = fast->next->next;
        slow = slow->next;
    }

    // If there are even elements in the list
    // then move fast
    if (fast->next->next == head) {
        fast = fast->next;
    }

    // Set the head pointer of the first half
    result.first = head;
    // Set the head pointer of the second half
    result.second = slow->next;
    // Make the second half circular
    fast->next = slow->next;
    // Make the first half circular
    slow->next = head;
    return result;
}

```

Split a Circular Linked List into two halves

Header List and Sentinel Node

A **header node** is a special node placed at the very beginning of a linked list. It usually does not store regular data like the other nodes. Instead, it may store useful information about the list, such as the total number of nodes or the minimum and maximum values. The pointer (link) in the header node points to the first actual data node in the list.

Header List and Sentinel Node

Sentinel Node:

A sentinel node is a special dummy node used as a marker to make algorithms simpler and avoid special-case checks (like checking for `null`). It does not store actual data.

In a doubly linked list, two sentinel nodes are often used—one at the beginning (header) and one at the end (trailer). These help make insertion and deletion easier.

In a circular linked list, a single sentinel node can act as both the head and the tail. When the list is empty, the sentinel's pointer simply points back to itself

Header List and Sentinel Node (key characteristics)

Simplifies the Code:

Sentinel or header nodes remove the need for special checks, such as testing whether the head is `NULL` before inserting or deleting a node. This makes insertion and deletion operations easier to implement.

Permanent Starting Point:

The header or sentinel node is always present in the list and is never removed. This guarantees that the list always has a fixed starting point.

Reduces Programming Errors:

Because the structure is consistent and edge cases are minimized, the chances of making mistakes in the code are lower. This makes the algorithms more reliable.

Can Improve Efficiency (in some cases):

A header node can store extra information about the list, such as its size. This allows quick access to that information without having to go through all the nodes.

Small Memory Cost:

The disadvantage is that sentinel or header nodes use a small amount of extra memory, even though they do not store actual data from the application.

Generalized Linked List

A **Generalized Linked List (GLL)** is a special type of linked list in which each node can store **either a single data element or another linked list**.

Anurag yadav, saksham sharma

—> late punch for consecutive days.

```
struct Node {  
    int tag; // 0 = data, 1 = sublist  
    union {  
        char data;  
        struct Node* sublist;  
    };  
    struct Node* next;  
};
```

```
// Main function
int main() {
    Node* head = NULL;

    // Create list: A (B C) D

    head = insert(head, createDataNode('A'));

    Node* sublist = NULL;
    sublist = insert(sublist, createDataNode('B'));
    sublist = insert(sublist, createDataNode('C'));

    head = insert(head, createSublistNode(sublist));
    head = insert(head, createDataNode('D'));

    printf("Generalized Linked List: ");
    display(head);

    freeGLL(head);
    return 0;
}
```

Generalized Linked List : main function

Generalized Linked List : Node creation

```
// Create data node
Node* createDataNode(char value) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->tag = 0;
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}
```

Generalized Linked List : sublist creation

```
// Create sublist node
Node* createSublistNode(Node* sublist) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->tag = 1;
    newNode->sublist = sublist;
    newNode->next = NULL;
    return newNode;
}
```

Generalized Linked List : insert at end

```
// Insert at end
Node* insert(Node* head, Node* newNode) {
    if (head == NULL)
        return newNode;

    Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    return head;
}
```

Generalized Linked List : display

```
// Display GLL (recursive)
void display(Node* head) {
    Node* temp = head;

    while (temp != NULL) {
        if (temp->tag == 0) {
            printf("%c ", temp->data);
        } else {
            printf("( ");
            display(temp->sublist);
            printf(") ");
        }
        temp = temp->next;
    }
}
```

Generalized Linked List : Free Memory

```
// Free memory (recursive)
void freeGLL(Node* head) {
    while (head != NULL) {
        Node* temp = head;
        if (head->tag == 1)
            freeGLL(head->sublist);
        head = head->next;
        free(temp);
    }
}
```


Skip List

A **skip list** is a probabilistic data structure that stores elements in **sorted order** and supports fast **search, insert, and delete**—on average **$O(\log n)$** time.

Level 0 is a normal sorted linked list (contains all elements).

Higher levels “skip” over nodes (contain fewer elements), like express lanes.

When you search, you move right on a high level to jump ahead, then drop down levels—similar to binary search but using linked lists.

The number of levels for each node is chosen **randomly**, which keeps the structure balanced *on average* without complex rotations (unlike AVL/Red-Black trees).

Skip List

```
typedef struct snode {  
    int key;  
    int value;  
    struct snode **forward;  
} snode;
```

`key` (for sorting)

`value` (payload)

`forward` = an **array of next pointers**, one per level

`forward[1]` = normal linked list next

`forward[2]` = “express lane” next

```
typedef struct skiplist {  
    int level;        // current highest level in use  
    int size;  
    struct snode *header;  
} skiplist;
```

`header` is a special node that serves as the start of every level

`level` is the highest active level currently in the structure

Skip list: Node Creation

```
skiplist *skiplist_init(skiplist *list) {  
    int i;  
    snode *header = (snode *) malloc(sizeof(struct snode));  
    list->header = header;  
    header->key = INT_MAX;  
    header->forward = (snode **) malloc(  
        sizeof(snode*) * (SKIPLIST_MAX_LEVEL + 1));  
    for (i = 0; i <= SKIPLIST_MAX_LEVEL; i++) {  
        header->forward[i] = list->header;  
    }  
  
    list->level = 1;  
    list->size = 0;  
  
    return list;  
}
```

So initially:

- Level 1: header → header (empty)
- Level 2: header → header (empty)

This is how they represent **NIL/end**.

This makes every forward pointer point back to the header itself.

So the list starts with only level 1 active.

```
static int rand_level() {  
    int level = 1;  
    while (rand() < RAND_MAX / 2 && level < SKIPLIST_MAX_LEVEL)  
        level++;  
    return level;  
}
```

This is coin-flip logic:

- 50% chance node stays at level 1
- 25% chance level 2
- 12.5% chance level 3 ...
So taller nodes are rarer →
that's what gives skip lists
 $O(\log n)$ average.

```

int skiplist_insert(skiplist *list, int key, int value) {
    snode *update[SKIPLIST_MAX_LEVEL + 1];
    snode *x = list->header;
    int i, level;
    for (i = list->level; i >= 1; i--) {
        while (x->forward[i]->key < key)
            x = x->forward[i];
        update[i] = x;
    }
    x = x->forward[1];

    if (key == x->key) {
        x->value = value;
        return 0;
    } else {
        level = rand_level();
        if (level > list->level) {
            for (i = list->level + 1; i <= level; i++) {
                update[i] = list->header;
            }
            list->level = level;
        }

        x = (snode *) malloc(sizeof(snode));
        x->key = key;
        x->value = value;
        x->forward = (snode **) malloc(sizeof(snode*) * (level + 1));
        for (i = 1; i <= level; i++) {
            x->forward[i] = update[i]->forward[i];
            update[i]->forward[i] = x;
        }
    }
    return 0;
}

```

Start at highest level

Move right while next key is still < key

When you can't move right, store that node in `update[i]`

Drop down to next lower level

Repeat

`update[i]` ends up being **the node after which your new node should be inserted at level i.**

Now `x` is the node that comes immediately after where the key would go at level 1.

if key already exists, So it updates value.

```

int skiplist_insert(skiplist *list, int key, int value) {
    snode *update[SKIPLIST_MAX_LEVEL + 1];
    snode *x = list->header;
    int i, level;
    for (i = list->level; i >= 1; i--) {
        while (x->forward[i]->key < key)
            x = x->forward[i];
        update[i] = x;
    }
    x = x->forward[1];

    if (key == x->key) {
        x->value = value;
        return 0;
    } else {
        level = rand_level();
        if (level > list->level) {
            for (i = list->level + 1; i <= level; i++) {
                update[i] = list->header;
            }
            list->level = level;
        }

        x = (snode *) malloc(sizeof(snode));
        x->key = key;
        x->value = value;
        x->forward = (snode **) malloc(sizeof(snode*) * (level + 1));
        for (i = 1; i <= level; i++) {
            x->forward[i] = update[i]->forward[i];
            update[i]->forward[i] = x;
        }
    }
    return 0;
}

```

otherwise
create a new
node

If the new node is
taller than current list
level:

So for new
levels, insertion
starts after
header.

This is standard linked list insertion at
multiple levels:

- new node points to what
update[i] was pointing to
- update[i] now points to new node

```

// Insert a key
void insert(SkipList* list, int key) {
    SkipNode* update[MAX_LEVEL + 1];
    SkipNode* x = list->header;

    // Find positions to update
    for (int i = list->level; i >= 0; i--) {
        while (x->forward[i] != NULL && x->forward[i]->key < key) {
            x = x->forward[i];
        }
        update[i] = x;
    }

    x = x->forward[0];

    // No duplicates (optional rule)
    if (x != NULL && x->key == key) return;

    int lvl = randomLevel();
    if (lvl > list->level) {
        for (int i = list->level + 1; i <= lvl; i++) {
            update[i] = list->header;
        }
        list->level = lvl;
    }

    SkipNode* newNode = createNode(lvl, key);
    for (int i = 0; i <= lvl; i++) {
        newNode->forward[i] = update[i]->forward[i];
        update[i]->forward[i] = newNode;
    }
}

```

```

typedef struct SkipNode {
    int key;
    struct SkipNode** forward; // array of forward pointers (size = level+1)
} SkipNode;

typedef struct SkipList {
    int level;           // current highest level
    SkipNode* header;    // header node
} SkipList;

```

```
// Search a key
SkipNode* search(SkipList* list, int key) {
    SkipNode* x = list->header;

    for (int i = list->level; i >= 0; i--) {
        while (x->forward[i] != NULL && x->forward[i]->key < key) {
            x = x->forward[i];
        }
    }

    x = x->forward[0];
    if (x != NULL && x->key == key) return x;
    return NULL;
}
```

```
int deleteKey(SkipList* list, int key) {
    SkipNode* update[MAX_LEVEL + 1];
    SkipNode* x = list->header;

    for (int i = list->level; i >= 0; i--) {
        while (x->forward[i] != NULL && x->forward[i]->key < key) {
            x = x->forward[i];
        }
        update[i] = x;
    }

    x = x->forward[0];
    if (x == NULL || x->key != key) return 0; // not found

    for (int i = 0; i <= list->level; i++) {
        if (update[i]->forward[i] != x) break;
        update[i]->forward[i] = x->forward[i];
    }

    free(x->forward);
    free(x);

    while (list->level > 0 && list->header->forward[list->level] == NULL) {
        list->level--;
    }

    return 1;
}
```

```
// Display level 0 (the full sorted list)
void display(SkipList* list) {
    SkipNode* x = list->header->forward[0];
    printf("SkipList (level 0): ");
    while (x != NULL) {
        printf("%d ", x->key);
        x = x->forward[0];
    }
    printf("\n");
}
```

```
// Free whole list
void freeSkipList(SkipList* list) {
    SkipNode* cur = list->header->forward[0];
    while (cur) {
        SkipNode* nxt = cur->forward[0];
        free(cur->forward);
        free(cur);
        cur = nxt;
    }
    free(list->header->forward);
    free(list->header);
    free(list);
}
```

```
// Demo main
int main(void) {
    srand((unsigned)time(NULL));

    SkipList* list = createSkipList();

    insert(list, 10);
    insert(list, 30);
    insert(list, 20);
    insert(list, 40);
    display(list);

    int k = 20;
    printf("Search %d: %s\n", k, search(list, k) ? "Found" : "Not Found");

    printf("Delete 30: %s\n", deleteKey(list, 30) ? "Deleted" : "Not Found");
    display(list);

    freeSkipList(list);
    return 0;
}
```